

CourtrightCollective

IDEA TO APP STORE

COURTRIGHT COLLECTIVE

How I Shipped 6 Apps *Solo*

The full playbook. Idea to App Store, working nights and weekends, with a family and a day job.

LAUNCH PLAYBOOK

6 PHASES

CHECKLISTS INCLUDED

COURTRIGHTCO.COM

BUILT IN NASHVILLE

How I Shipped 6 Apps Solo

Before we start

I am not going to tell you shipping an app is easy, and I am not going to tell you it changed my life. It is a lot of nights. It is a lot of 9pm to 1am after the kids are down. Some of it is genuinely fun and some of it is fighting a provisioning profile at midnight for reasons nobody has ever fully explained.

But here's the thing. The reason most people never ship is not talent and it is not time. It is that they have no process, so every single decision is a fresh decision, and fresh decisions at 11pm are how projects die.

This is my process. Six phases, each with a clear finish line. When I am tired and I do not know what to do next, I open this and do the next thing on the list. That is the whole trick. There is no other trick.

Who this is for: you can already build something, or you are willing to learn as you go. You have limited hours. You want a finished thing in an app store, not a portfolio piece.

Phase I: Pick something small enough to finish

The only rule that matters

If you cannot describe the app in one sentence, you cannot finish it in your available hours.

Not "a social app for people who like cooking with a marketplace and a video feed." That is four apps. One sentence means one sentence.

Test yours. Say it out loud right now. If it needs an "and," cut everything after the "and" and put it in a file called `version-two.md`. That file is where good ideas go to wait patiently.

The scoping questions

Answer all six honestly before you write any code.

QUESTION	WHY IT MATTERS
Can one person build v1 in under 80 hours?	80 hours is roughly 10 weeks of nights. Past that, life happens and momentum dies
Does it need a backend?	A backend doubles your build and adds forever-costs. Avoid it for v1 if you possibly can
Does it need other people to be useful?	Anything social needs a crowd. You do not have a crowd yet
Can I make the content myself?	If v1 needs 500 items of anything, that is not a coding project, it is a content project wearing a coding hat
Would I use this weekly?	If not, you will not survive month three of maintenance
Can I describe who wants this in one specific sentence?	"Everyone" means nobody, and it makes every marketing decision impossible later

Score six out of six and build it. Score four, cut scope until you score six. Score two, pick a different idea and do not feel bad about it, that decision just saved you three months.

The scope cut nobody wants to make

Every v1 I have shipped got cut roughly in half between the idea and the submission. Every single one. The cut always felt like ruining the app. It never once did.

Things that felt essential and were not:

- User accounts (local storage got me to launch on more than one app)
- Onboarding tutorials (a well-designed first screen did the job)
- Settings screens (I shipped with three settings, nobody asked for more)
- Dark mode (add it in 1.1, people love an update)

- Android at launch (one store at a time, always)
- Sharing features (nobody shares an app they used once)

Things that were genuinely essential:

- The core loop working perfectly, every time, on every device
- Not crashing
- Looking like somebody cared

That is it. That is the list.

Phase 1 finish line

- [] One sentence written down
 - [] Six scoping questions answered honestly
 - [] `version-two.md` created and already has things in it
 - [] The core loop sketched on paper, screen by screen
 - [] A name you can live with (see Phase 5, do not spend three weeks here)
-

Phase 2: Validate before you build

Two weeks, maximum

Validation for indie apps does not mean surveys and focus groups. It means finding out whether people are already trying to solve this problem and hating how they do it.

Do these five things. Cap it at two weeks.

- 1. Search the store for your idea.** If there are zero competitors, that is usually bad news, not good news. It means no demand. If there are forty, look at the ratings.
- 2. Read the one and two star reviews of the top five competitors.** This is the single most valuable hour in the entire process. People will tell you exactly what is broken and exactly what they wish existed. Copy their phrases into a file. Those phrases become your marketing copy later.
- 3. Find where the people are.** A subreddit, a Facebook group, a Discord, a forum. Read for a week without posting anything. What do they complain about repeatedly?
- 4. Ask ten real people the problem question, not the product question.** "How do you currently handle [problem]?" not "would you use an app that [solution]?" Everyone says yes to the second question. Nobody's yes means anything.
- 5. Check whether anyone is spending money.** Are the competitors charging? Are people paying? A category where everything is free and ad-supported is a category where you will not make money.

The kill criteria

Write these down before you start looking, so you cannot move the goalposts on yourself:

- Nobody in the community mentions this problem unprompted → kill it
- The top competitors have 4.7 ratings and thousands of reviews → pick a narrower angle or kill it
- You cannot find ten people who have the problem → kill it
- Everything in the category is free and nobody upgrades → build it for fun, not for money, and be honest with yourself about which one you are doing

Killing an idea in week two is a win. It is the cheapest possible outcome for a bad idea.

Phase 2 finish line

- [] Competitor review file with at least 20 real complaints, in customers' own words
 - [] Ten problem conversations done
 - [] One narrow angle chosen: who exactly this is for, and what the existing options get wrong
 - [] Kill criteria checked, and you decided to proceed on purpose
-

Phase 3: Build the thing

Set the schedule before you need it

Motivation gets you through week one. Schedule gets you through week seven.

What works: the same blocks every week, written on the calendar as if they were meetings someone else scheduled. Mine are nights after bedtime and one weekend morning. Yours will be different. What matters is that they are decided in advance, so at 9pm you are not negotiating with yourself.

Protect two things ruthlessly: the block itself, and the first ten minutes of the block. If you spend the first ten minutes figuring out what you were doing, you have lost a quarter of the session. Which is why the next rule exists.

End every session with the next step written down

Before you close the laptop, write one line: **the exact next thing to do.**

Not "work on settings screen." That is a decision you will have to make again tomorrow while tired. Write: "add the toggle for sound in SettingsView, wire it to UserDefaults, key is `soundEnabled`."

This one habit is worth more than any productivity system I have tried. It converts a cold start into a warm one, every session, forever.

The build order that keeps you sane

- 1. The core loop, ugly.** No design, no polish. Just the thing the app does, working end to end. If the core loop is not fun or useful when it is ugly, no amount of design will save it. Find that out in week one, not week nine.
- 2. Real data.** Get actual content in there early. Placeholder data lies to you about how things look and feel.
- 3. The empty states.** First launch, no data, no connection, error. Every app I have shipped, this is where the roughness hides. Do it early while you still care.
- 4. Design pass.** Now make it look like somebody cared. Spacing, type, color, one consistent accent. This is also when the app starts feeling real and your motivation refills.
- 5. Edge cases and devices.** Small phone, big phone, oldest OS you support, accessibility text sizes, rotation if you allow it.
- 6. Polish.** Haptics, transitions, sounds, the small delights. Save this for last. It is the reward for finishing, and it is also the first thing to cut if you run out of time.

The rules that saved me the most time

- **Ship one platform first.** iOS or Android, pick one. Doing both at once does not double your work, it triples it.

- **Use the platform's own components.** Custom UI is where solo timelines go to die. Native components are free, accessible, and update themselves.
- **No backend for v1 if you can avoid it.** Local storage, on-device data, or a static file. A backend adds auth, hosting, migrations, downtime, and a monthly bill for an app with four users.
- **Third-party dependencies are a loan.** Every one you add is a thing that can break during someone else's update cycle. Take a few. Read the license. Do not take twelve.
- **Test on a real device from week one.** The simulator lies about performance, gestures, and how big your text really is.
- **Commit every session.** Even broken work, on a branch. The night your machine dies is not the night to learn this.

When you stall

Every project has a week where you do not want to open it. Not a crisis, just flat. Three things that work:

1. **Do a 15 minute task.** Fix a typo, adjust a color, delete dead code. Momentum is real and it starts small.
2. **Use the app like a user for five minutes.** You will immediately find three things to fix and remember why you started.
3. **Show it to one person.** Not for feedback. For the feeling of someone seeing it. That works better than it has any right to.

Phase 3 finish line

- ☐ Core loop works on a real device with real data
 - ☐ Every empty and error state handled
 - ☐ Tested on the oldest supported OS and the smallest screen
 - ☐ Zero known crashes
 - ☐ Accessibility: largest text size does not break your layouts
 - ☐ Analytics installed and firing, with the three events that actually matter
 - ☐ Crash reporting installed and verified with a deliberate test crash
-

Phase 4: The submission gauntlet

Budget a full week for this. Not because it takes a week of work, but because something always comes back and you do not want that something landing on your launch date.

Everything you need before you can submit

Accounts and legal

- ☐ Developer account paid and active (Apple runs annually, Google is one-time)
- ☐ Bank and tax forms completed and accepted, this takes longer than you expect
- ☐ Privacy policy written and hosted at a real URL
- ☐ Terms of service if you have accounts or purchases
- ☐ Support URL or support email that you actually monitor

In the app

- ☐ App icon at every required size, no transparency, no rounded corners baked in
- ☐ Launch screen
- ☐ Version and build numbers set correctly
- ☐ Purchases tested in sandbox, including restore
- ☐ Restore purchases button exists and works, this is a common rejection
- ☐ Account deletion path if you have accounts, this is required
- ☐ No debug UI, no test data, no console spam

Store listing

- ☐ Name, subtitle, keywords, description (see the ASO Cheat Sheet)
- ☐ Screenshots for every required size
- ☐ Privacy nutrition labels answered accurately, including third-party SDKs
- ☐ Age rating questionnaire completed
- ☐ Category chosen, primary and secondary
- ☐ Demo account credentials in review notes if anything is behind a login

The five rejections you will probably get

REJECTION	ACTUAL FIX
Guideline 2.1, more information needed	Reviewer could not access something. Add demo credentials and a written walkthrough in the notes
Missing restore purchases	Add the button. It is required, not optional
Privacy policy link broken or too vague	Real URL, publicly accessible, actually describes what you collect
Metadata does not match the app	Screenshots or description promise something the build does not do
Account deletion missing	If users can create an account, they must be able to delete it in-app

When you get rejected: do not argue in the first reply. Read what they asked, fix it precisely, resubmit with a short note explaining what changed. Most rejections clear in one round. If a rejection is genuinely wrong, you can escalate, but that costs days, so only spend them on something that matters.

Phase 4 finish line

- [] Build approved
 - [] Release set to manual, not automatic. You want to choose your launch day
 - [] Everything in Phase 5 ready before you press the button
-

Phase 5: Launch

Pick your day

Tuesday or Wednesday morning. Not Friday. Ship on a Friday and you will spend your Saturday fixing something while everyone else in your house is doing something better.

The two weeks before

- ☐ Landing page live, one page is plenty, with a clear store link
- ☐ Screenshots and a short clip exported for social
- ☐ Announcement written for every place you will post
- ☐ Five social posts drafted, so launch week does not need fresh writing
- ☐ Everyone who asked to be told, told (a personal message beats a broadcast every time)
- ☐ Support email set up and tested
- ☐ A short FAQ ready for the questions you already know are coming

Launch day, in order

1. Release the build. Confirm it is actually live in your own store region.
2. Check the store page on a phone. Not a browser. A phone.
3. Post the announcement everywhere at once. Do not stagger it, you only get one day one.
4. Message people individually. Ten personal messages outperform one broadcast post, every time.
5. Watch crash reporting for four hours. This is the window where anything catastrophic shows up.
6. Reply to every single review and comment.
7. Stop refreshing the dashboard. Go do something else. Seriously.

What launch day actually looks like

Manage this expectation now: launch day for an indie app with no audience is quiet. Dozens of downloads, not thousands. That is normal. That is not failure.

The apps that grow do it in months, from search traffic, word of mouth, and updates. Launch day is not the finish line. It is the day the clock starts.

Phase 5 finish line

- ☐ Live in every intended region
 - ☐ Announced everywhere
 - ☐ Zero crashes in the first 24 hours
 - ☐ Every day-one review replied to
-

Phase 6: The part everyone skips

Most indie apps do not fail at launch. They fail in month two, when the developer moves on to the next idea and the app sits there quietly rotting.

The first 30 days

Week 1. Reply to everything. Fix any crash immediately, no matter how small. Watch where people drop off in your analytics and be honest about what you see.

Week 2. Ship a small update. Even a tiny one. It signals life to the store and to users, and it gets you a fresh What's New entry.

Week 3. Look at your day 1 and day 7 retention. This is the number that determines whether you have a product or a project. If day 7 retention is under 10 percent, stop marketing and go fix the product.

Week 4. Write down three things that went wrong and one thing that went better than expected. Put it in your Ship Log. Then decide, deliberately: keep pushing this one, or start the next.

The maintenance rhythm that keeps an app alive

CADENCE	DO THIS
Weekly	Reply to reviews, check crash rate
Every 4 to 8 weeks	Ship an update, however small
Monthly	Record your metrics: downloads, actives, revenue, retention, rating
Quarterly	Refresh keywords and screenshots
Yearly	Decide honestly whether this app still deserves your hours

Knowing when to stop

Not every app should live forever, and quitting the right one is not failure. Signs it is time:

- Day 7 retention stays under 5 percent after two real attempts at fixing it
- You have shipped three meaningful updates and nothing moved
- Maintenance costs more per month than it earns, and you no longer enjoy it
- You would not build it again today

You can leave it live and stop investing. You can hand it off. You can pull it. All three are legitimate. What is not legitimate is letting it quietly drain the hours you should be spending on the next thing.

The six lessons, condensed

1. **Cut the scope in half.** Then cut it again. Every app I shipped got cut roughly in half and none of them suffered for it.
2. **Write down the next step before you close the laptop.** Cheapest productivity habit that exists.
3. **The ugly core loop comes first.** If it is not good ugly, it will not be good pretty.
4. **Ship one platform, one store, one audience.** Focus is the only real advantage a solo dev has.
5. **Launch day is quiet, and that is fine.** The growth comes from month three, search traffic, and updates.
6. **Reply to every review.** It is the highest-return twenty minutes in your entire week.

And the one under all of them: **finished and imperfect beats perfect and unreleased, every single time.** Nobody has ever downloaded the app you are still polishing.

The whole thing on one page

PHASE	TIME	FINISH LINE
1. Pick	1 week	One sentence, six-out-of-six score, core loop sketched
2. Validate	2 weeks	20 real complaints, 10 conversations, one narrow angle
3. Build	6 to 10 weeks	Core loop solid, no crashes, empty states handled
4. Submit	1 week	Approved, release set to manual
5. Launch	1 day + 2 weeks prep	Live, announced, every review answered
6. Keep going	Forever	Update every 4 to 8 weeks, metrics logged monthly

Roughly 12 to 16 weeks of nights. That is one app a quarter if you want it badly enough, and honestly, one a year is a perfectly good pace for a person with a life.

Go pick something small. Write the sentence. Start Phase 1 tonight.

courtrightco.com